Informe por actividades · Unidad 7

NoSQL y nuevas formas de almacenamiento — Food Store

Tecnicatura Universitaria en Programación · Bases de Datos II · Comisión 4
Integrante: Facundo Chacón · Fecha: 03/11/2026
Repositorio: github.com/VIKROOM/DB-TPS-COM4
PostgreSQL 17 (base foodstore) · MongoDB 7.0.14 + mongosh 2.3.1

Este documento desglosa el trabajo en las siete actividades que se derivan de las consignas 1 a 7 y de la entrega. Para cada actividad se indica qué pedía la consigna, qué se hizo, con qué evidencia se comprobó y cuál fue el resultado. El PDF de entrega de 2 páginas es el resumen exigido; este informe es el detalle operativo de cada paso.

Mapa de actividades

#	Actividad	Consigna	Evidencia principal	Estado
A1	Modelo documental y correspondencia	1	Sección 1 del informe; documento de ejemplo	**OK**
A2	Base, colección y migración	2	01_crear_coleccion.js, 02_datos_pedidos.js	**OK**
A3	Reglas de integridad y embebido	3	03_verificacion_integridad.js (9 controles)	**OK**
A4	Consultas a–d en MongoDB	4	consultas_mongodb.js y salida real	**OK**
A5	Equivalentes en PostgreSQL	5	consultas_postgresql.sql y tabla comparativa	**OK**
A6	IA y contraste de afirmaciones	6 y 7	BITACORA_IA.md y pruebas de laboratorio	**OK**
A7	Conclusión, PDF y entrega	7 y Entrega	PDF de 2 páginas y 26/26 comprobaciones	**OK**

Actividad 1 · Modelo documental y correspondencia (consigna 1)

Consigna. Seleccionar el subconjunto «pedidos y detalle de pedidos», identificar las tablas y claves de PostgreSQL y elaborar un esquema breve de la colección documental.

El subconjunto elegido son los pedidos con sus líneas y los datos esenciales del cliente. En PostgreSQL intervienen cuatro tablas: pedido (PK id, FK cliente_id, fecha, forma_pago), linea_pedido (PK compuesta pedido_id, producto_id, cantidad, precio_unitario), cliente (PK id, nombre, apellido) y producto (PK id, FK categoria_id, nombre, precio). Las FK de linea_pedido usan CASCADE hacia el pedido y RESTRICT hacia el producto; pedido.cliente_id usa RESTRICT.

Cada pedido se representa como un único documento con las líneas embebidas:

```
{
  pedido_id: 1,  fecha: ISODate("2026-11-03T01:39:32.658Z"),
  estado: "PENDIENTE",  forma_pago: "EFECTIVO",
  cliente: { cliente_id: 1, nombre: "Ana Gomez" },
  items: [
    { producto_id: 1, nombre: "Muzzarella", categoria: "Pizzas",
      cantidad: 2, precio_unitario: NumberDecimal("1000.00"),
      subtotal: NumberDecimal("2000.00") },
    { producto_id: 2, nombre: "Coca 1.5L", categoria: "Bebidas",
      cantidad: 1, precio_unitario: NumberDecimal("800.00"),
      subtotal: NumberDecimal("800.00") } ],
  items_count: 2,  total: NumberDecimal("2800.00")
}
```

Correspondencia. pedido.id → pedido_id; fecha → fecha; forma_pago → forma_pago; cliente.id → cliente.cliente_id; nombre + apellido → cliente.nombre; linea_pedido + producto → items[]; SUM(cantidad × precio_unitario) → total. Se conservan pedido_id, cliente_id y producto_id; no se copian email ni teléfono (minimización de datos).

Decisión de agrupamiento. Las líneas se embeben porque siempre se leen junto con el pedido, el arreglo es acotado (2 líneas por pedido en la muestra) y la escritura del pedido completo queda atómica. Cliente y producto se referencian por id y solo se copia la instantánea histórica necesaria (nombre, categoría y precio unitario).

Evidencia y verificación. El documento de ejemplo se reproduce en la sección 1 del informe de entrega y en evidencia/mongo_06_pruebas_ia.txt. Los 12 documentos migrados conservan los tres identificadores de origen y ninguno incluye email o teléfono.

Actividad 2 · Base, colección y migración (consigna 2)

Consigna. Crear la base y la colección en MongoDB y migrar un mínimo de cinco pedidos con sus líneas y datos esenciales del cliente mediante insertMany() o equivalente.

La migración se resolvió en tres pasos reproducibles: (1) sql/01_exportar_pedidos_mongodb.sql consulta PostgreSQL y genera un script JavaScript con ISODate y NumberDecimal; (2) mongodb/01_crear_coleccion.js crea foodstore_nosql.pedidos con validador $jsonSchema, índices y metadatos de procedencia; (3) mongodb/02_datos_pedidos.js ejecuta el insertMany().

Dato migrado	Valor verificado
Volumen migrado	12 pedidos · 24 líneas · 11 clientes (mínimo exigido: 5 pedidos)
Rango de fechas	2025-03-01 a 2026-11-02
Importe total	$560.979,00 (idéntico a PostgreSQL)
Campos por documento	pedido_id, fecha, estado, forma_pago, cliente, items, items_count, total

Validador e índices. El validador exige los ocho campos, tipos BSON correctos, estado y forma_pago dentro de su dominio, cantidad entero ≥ 1, importes decimales ≥ 0, items no vacío y additionalProperties: false. Se crearon índices por pedido_id (único), estado, items.producto_id (multikey) y total (descendente). Los metadatos quedan en meta_migracion con origen, reglas y fecha de extracción.

Evidencia y verificación. evidencia/mongo_02_carga.txt registra Documentos en pedidos: 12; evidencia/mongo_01_crear_coleccion.txt muestra los cuatro índices creados. Los 12 documentos contienen fecha, estado, cliente, productos, cantidades y precios históricos.

Actividad 3 · Reglas de integridad y justificación del embebido (consigna 3)

Consigna. Indicar dos reglas de integridad que deben conservarse en la migración y justificar qué datos se incluyeron dentro del documento en lugar de colecciones separadas.

Regla	Implementación	Resultado medido
1. Identidad y referencia	pedido_id obligatorio y único; cada items.producto_id debe existir en producto del origen.	0 duplicados de pedido_id; 0 líneas huérfanas sobre 24 líneas.
2. Coherencia de cantidades e importes	cantidad entero ≥ 1; precio_unitario ≥ 0; subtotal = cantidad × precio_unitario; total = Σ subtotales.	0 subtotales incorrectos en 24 líneas; 0 totales incorrectos en 12 pedidos.

Por qué se embebe. Las líneas se leen siempre junto con el pedido, son un arreglo acotado y permiten recuperar el pedido completo en una sola lectura, sin JOIN. El cliente y el producto quedan como referencia por id: no se duplica el catálogo ni los datos personales, y se conserva la trazabilidad al modelo relacional. La copia de nombre, categoría y precio unitario es una instantánea histórica: 23 de 24 precios históricos difieren del precio actual del catálogo, lo que confirma que no se sobrescriben.

Límite asumido. El embebido es válido para arreglos acotados; un documento BSON tiene un máximo de 16 MiB. Para un histórico ilimitado de líneas correspondería una colección separada con referencia al pedido. En esta muestra cada pedido tiene 2 líneas.

Evidencia y verificación. mongodb/03_verificacion_integridad.js ejecuta 9 controles en el motor (documentos, duplicados, items_count, subtotales, totales, dominio de estado, tipo de fecha, tipos BSON e ids de producto). Salida: RESULTADO: OK en evidencia/mongo_03_verificacion_integridad.txt.

Actividad 4 · Consultas a–d en MongoDB (consigna 4)

Consigna. Ejecutar cuatro consultas adaptadas al modelo: todos los pedidos con identificador y estado; filtrar por un estado existente; buscar un producto dentro del arreglo de líneas; y buscar totales superiores a un importe elegido, justificando el umbral.

#	Consulta ejecutada (resumen)	Resultado sobre los 12 documentos
a	find({}, {_id:0, pedido_id:1, estado:1}).sort({pedido_id:1})	12 pedidos, con pedido_id y estado
b	find({estado:"TERMINADO"}, {_id:0, pedido_id:1})	11 pedidos: ids 2 a 12
c	find({"items.producto_id": 1}, {_id:0, pedido_id:1})	2 pedidos: ids 1 y 2 (Muzzarella)
d	find({total: {$gt: NumberDecimal("50000.00")}})	6 pedidos: 2, 6, 8, 10, 11 y 12 · $386.471,00

Justificación del umbral de d): $50.000,00. Sobre los 12 pedidos, el total promedio es $46.748,25 y la mediana $49.891,50. El umbral elegido queda en el orden de la mediana y separa 6 de 12 pedidos de alto valor: no devuelve todos ni ninguno, así que la consulta discrimina. El importe acumulado de esos 6 pedidos es $386.471,00.

La distribución por estado de la muestra es 11 pedidos TERMINADO ($558.179,00) y 1 PENDIENTE ($2.800,00). La regla de estado se deriva de los datos porque el esquema real no tiene columna de estado: fecha_corte = max(fecha del lote) + 1 día; TERMINADO si fecha ≤ fecha_corte − 30 días. No depende del reloj.

Evidencia y verificación. consultas_mongodb.js contiene las cuatro consultas y las comprobaciones complementarias; la salida completa está en evidencia/mongo_04_consultas.txt.

Actividad 5 · Equivalentes en PostgreSQL y comparación (consigna 5)

Consigna. Resolver en PostgreSQL las consultas equivalentes a b) y c), comparar en una tabla la cantidad y los identificadores recuperados en ambos motores y explicar cualquier diferencia.

En SQL, b) se resuelve con un CASE que aplica exactamente la misma regla de estado y c) con un JOIN a linea_pedido más DISTINCT. Ambos motores trabajan sobre el mismo lote (pedido.id BETWEEN 1 AND 12) y la misma precisión decimal (NUMERIC(10,2) ↔ Decimal128).

Consulta	Cant. PostgreSQL	Cant. MongoDB	IDs en ambos motores	Importe	¿Coinciden?
b) estado TERMINADO	11	11	2,3,4,5,6,7,8,9,10,11,12	$558.179,00	**Sí**
c) producto 1	2	2	1,2	$79.810,00	**Sí**

Diferencias detectadas: ninguna en los datos. Cantidades, IDs e importes coinciden exactamente. La única diferencia es de sintaxis y modelo: SQL usa JOIN, CASE y agregación; MongoDB usa notación de punto sobre el arreglo y un campo estado materializado al migrar. El total general también coincide: $560.979,00 en ambos motores.

Evidencia y verificación. consultas_postgresql.sql incluye los equivalentes, la tabla de verificación y cuatro controles de integridad del origen; la salida está en evidencia/postgres_01_consultas.txt. La comparación de cantidad, IDs e importes se recalcula automáticamente en verificar_entrega.ps1 (8 comprobaciones cruzadas).

Actividad 6 · Consulta a IA y contraste de afirmaciones (consignas 6 y 7)

Consigna. Consultar a una herramienta de IA comparando PostgreSQL y MongoDB para el modelo actual, el acceso frecuente a pedidos completos y la coherencia de cantidades y precios; contrastar dos afirmaciones con documentación oficial o laboratorio y concluir en hasta 200 palabras con al menos dos criterios técnicos.

El prompt completo está transcripto en BITACORA_IA.md. La IA propuso embeber las líneas y formuló cuatro afirmaciones que se contrastaron con documentación oficial y con pruebas ejecutadas en el laboratorio:

Afirmación de la IA	Contraste	Veredicto
«No hay transacciones ACID multidocumento»	MongoDB las soporta en replica sets; un standalone no. En el laboratorio la sesión falló con Transaction numbers are only allowed on a replica set member or mongos.	Imprecisa: falsa en general, cierta en este despliegue. El diseño no las necesita.
«Es más rápido porque evita JOIN»	El embebido reduce operaciones, no garantiza superioridad. Consulta c): MongoDB examinó 2 documentos y 2 claves (1 ms); PostgreSQL usó index-only scans (0,086 ms).	No sostenible como absoluto: la ventaja es de número de operaciones.
«No se pueden imponer reglas de integridad»	El validador $jsonSchema con validationAction:"error" rechazó un documento con cantidad: 0: Document failed validation.	Falsa: el esquema flexible admite validaciones estrictas.
«El documento puede crecer sin límite»	El máximo BSON es 16 MiB; el embebido se recomienda para arreglos acotados.	Falsa: los pedidos migrados tienen 2 líneas y ~1,5 KB.

Conclusión (196 palabras). Conviene una arquitectura combinada, no un reemplazo. PostgreSQL debe seguir siendo la fuente de verdad: las claves foráneas garantizan que cada pedido tenga un cliente y productos existentes, impone cantidades positivas y conserva el precio histórico; además ofrece transacciones ACID en el despliegue real. MongoDB aporta una proyección de lectura: el pedido completo se recupera en un solo documento, sin JOIN, y el laboratorio mostró resultados idénticos a SQL. Esa ventaja es de número de operaciones, no de rendimiento demostrado con 12 documentos. Los criterios son (1) patrón de acceso —lectura completa y repetida frente a escrituras transaccionales—, (2) integridad —coherencia de cantidades, precios y referencias— y (3) costo operativo de mantener dos motores. Si el volumen de lectura lo justifica, sincronizar por pedido_id una colección de solo lectura.

Evidencia y verificación. BITACORA_IA.md documenta el prompt, la respuesta y la tabla de contraste; mongodb/06_pruebas_afirmaciones_ia.js ejecuta las pruebas de validador, transacción y explain (evidencia/mongo_06_pruebas_ia.txt).

Actividad 7 · Conclusión, PDF y entrega (consigna 7 y Entrega)

Consigna. Presentar un único PDF de hasta cinco páginas, sin contar el código, con modelo, evidencias, tabla de verificación, contraste de IA y conclusión; adjuntar consultas_mongodb.js y consultas_postgresql.sql.

Entregable	Archivo	Verificación
Informe de entrega	informe/Informe_TP_U7_NoSQL_FoodStore.pdf	2 páginas (límite 5), con las cinco secciones
Consultas MongoDB	consultas_mongodb.js	Ejecuta a–d y comprobaciones extra
Consultas PostgreSQL	consultas_postgresql.sql	Ejecuta b–c y controles de integridad
Informe por actividades	informe/Informe_TP_U7_Actividades.pdf	Este documento
Evidencias	evidencia/	Salidas reales de ambos motores y del verificador

Autoevaluación. verificar_entrega.ps1 ejecuta 26 comprobaciones: presencia de los 10 archivos exigidos, 12 pedidos y 24 líneas de origen, 0 huérfanas, 0 líneas inválidas, 12 documentos migrados, comparación cruzada de cantidad, IDs, importes y productos entre ambos motores, integridad del modelo documental y límite de páginas del PDF. Resultado final: 26/26 OK, 0 fallas (evidencia/verificacion_entrega.txt).

Reproducción completa desde la raíz del TP:

```
$env:PGPASSWORD="***"
.\reproducir.ps1 `
  -Psql    "C:\Program Files\PostgreSQL\17\bin\psql.exe" `
  -Mongosh "C:\...\mongosh.exe"

.\verificar_entrega.ps1 `
  -Psql    "C:\Program Files\PostgreSQL\17\bin\psql.exe" `
  -Mongosh "C:\...\mongosh.exe"    # 26/26 OK
```

Cierre y enlaces

Repositorio del trabajo: https://github.com/VIKROOM/DB-TPS-COM4
El TP completo (código, evidencias, informes y scripts de reproducción) está en la carpeta TP7_U7_NoSQL_FoodStore/ de ese repositorio. Las evidencias se regeneran con reproducir.ps1 y se validan con verificar_entrega.ps1.

Límites declarados. El estado es derivado porque el esquema relacional no lo tiene; la muestra es de 12 pedidos y no permite conclusiones de rendimiento; el mongod local es standalone, por lo que no admite transacciones multidocumento. El informe no afirma que MongoDB reemplace a PostgreSQL: propone combinar ambos motores según el patrón de acceso y el volumen.